

BANKING SYSTEM

BACKEND ENGINEERING & SECURITY

1. Executive Overview

The Banking System backend demonstrates a maintainable Controller-Service-Repository architecture for customer and account management. It exposes REST APIs, persists data in MySQL and protects application resources using Spring Security and JWT.

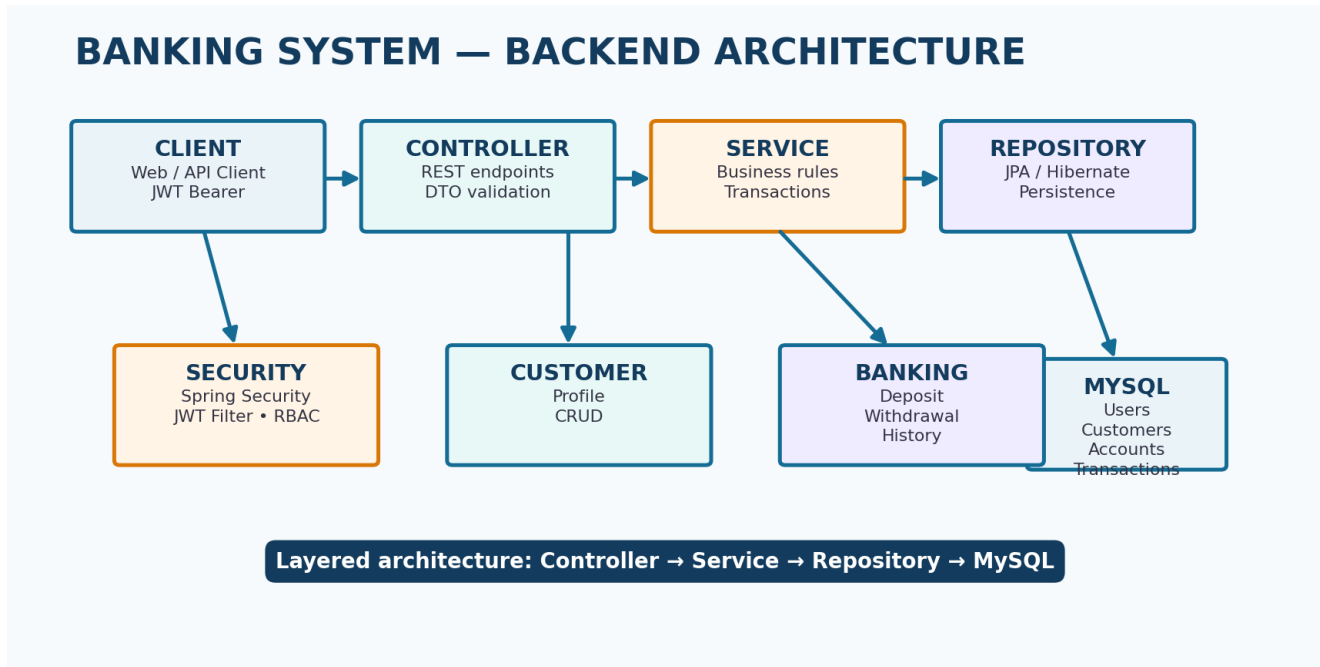
2. Scope & Contribution Boundary

- Controller-Service-Repository architecture
- Customer profile management
- Account management
- Deposit and withdrawal business logic
- Transaction history
- JWT authentication and authorization
- CUSTOMER / ADMIN RBAC
- Validation and centralized errors
- JUnit testing

3. Technology Stack

Area	Technology
Language	Java 21
Framework	Spring Boot 3.5.x
API	Spring Web / REST
Security	Spring Security + JWT
Persistence	Spring Data JPA / Hibernate
Database	MySQL 8
Testing	JUnit 5
Build	Maven

4. System Architecture

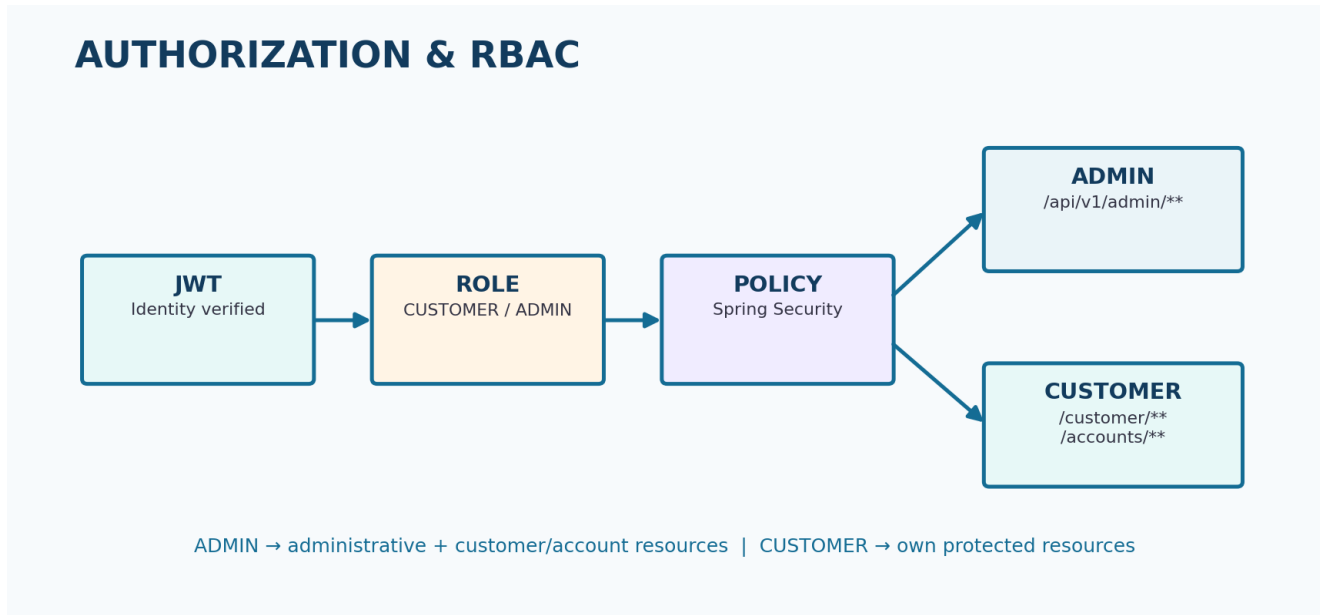


HTTP requests are handled by controllers, business rules reside in services, persistence is isolated in repositories, and MySQL stores the application state. JWT security is evaluated before protected operations.

5. Repository Structure

```
banking-system-backend/  
├── src/main/java/com/banking/system/  
│   ├── config/  
│   ├── controller/  
│   ├── dto/  
│   ├── entity/  
│   ├── enums/  
│   ├── exception/  
│   ├── repository/  
│   ├── security/  
│   └── service/  
├── src/test/java/com/banking/system/  
├── docs/assets/  
├── pom.xml  
├── .env.example  
├── .gitignore  
└── README.md
```

6. Authentication & Authorization

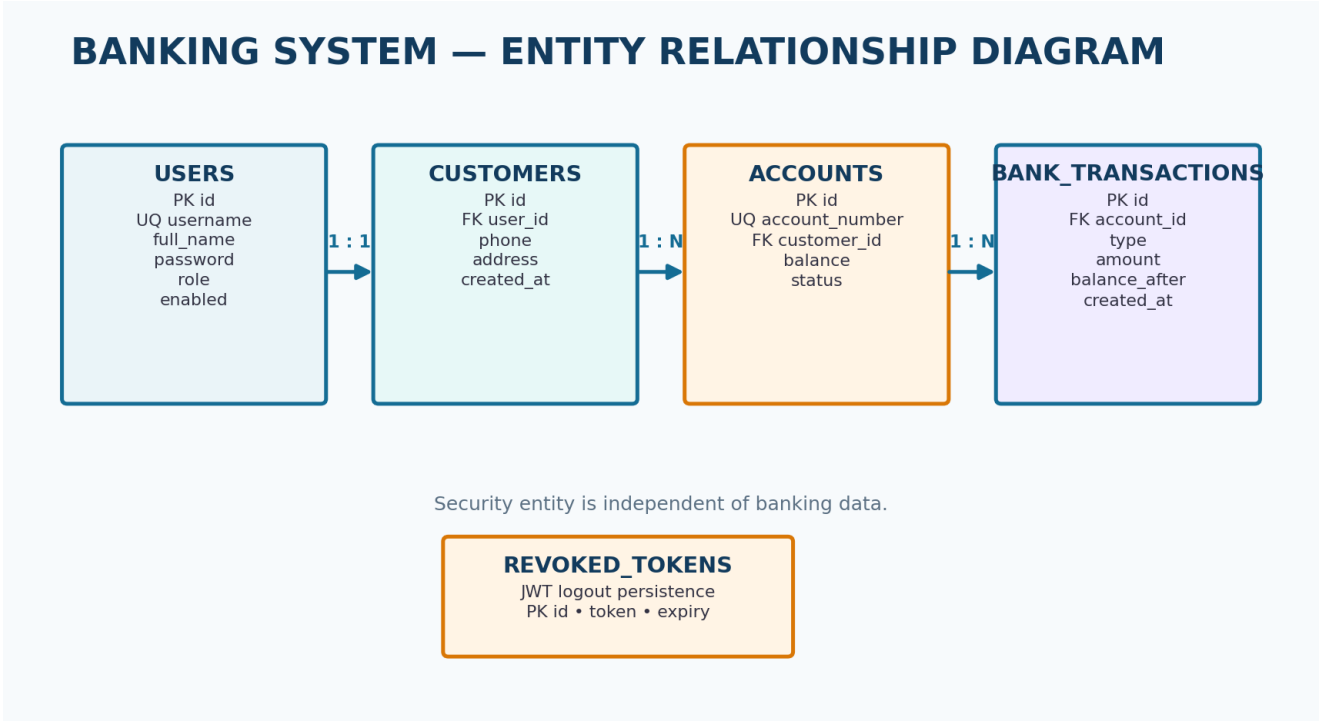


The authentication boundary validates JWT credentials and maps user roles to Spring Security authorities. CUSTOMER users receive customer/account access while ADMIN users can additionally access administrative resources.

7. Authentication Sequence

1. Client submits credentials.
2. AuthenticationManager validates credentials.
3. User identity is loaded from MySQL.
4. BCrypt verifies the password hash.
5. JWT is signed with an expiration time.
6. Protected requests send Authorization: Bearer <JWT>.
7. JWT filter validates signature, expiry and revocation before creating the SecurityContext.

8. Entity Relationship Diagram



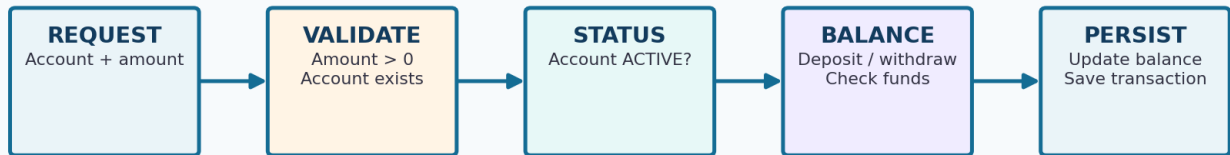
The core model connects users to customer profiles, customers to accounts, and accounts to immutable transaction records. Revoked tokens are stored separately for JWT logout enforcement.

Entity	Key Data	Relationship
users	id, username, password, role, enabled	1:1 Customer
customers	id, user_id, phone, address	1:N Account
accounts	id, account_number, balance, status	1:N Transaction
bank_transactions	account_id, type, amount, balance_after	Belongs to Account
revoked_tokens	token, expires_at, revoked_at	Security / logout

9. Banking Operation Flow

BANKING OPERATION FLOW

Deposit / Withdrawal business logic



Withdrawal with insufficient funds → reject without transaction persistence.

Result: new balance + immutable transaction record

8. Validate the requested amount.
9. Verify the account exists and is ACTIVE.
10. For withdrawal, verify sufficient balance.
11. Calculate the new balance.
12. Persist the account balance and transaction record.
13. Return the resulting balance and transaction details.

10. REST API Contract

Method	Endpoint	Access	Purpose
POST	/api/v1/auth/register	PUBLIC	Register customer
POST	/api/v1/auth/login	PUBLIC	Authenticate / JWT
POST	/api/v1/auth/logout	JWT	Revoke token
GET/PUT	/api/v1/customer/me	CUSTOMER	Profile
POST	/api/v1/accounts	AUTH	Create account
GET	/api/v1/accounts/{number}	AUTH	Account details
POST	/api/v1/accounts/{number}/deposit	AUTH	Deposit
POST	/api/v1/accounts/{number}/withdraw	AUTH	Withdrawal
GET	/api/v1/accounts/{number}/transactions	AUTH	History
POST	/api/v1/admin/accounts	ADMIN	Admin account creation

11. Business Rules

- Amounts must be greater than zero.
- Only ACTIVE accounts may transact.
- Withdrawals cannot make the balance negative.
- Every successful operation creates a transaction record.
- JWT-protected resources require a valid authenticated identity.

- Role checks restrict administrative resources.

12. Validation & Error Handling

- Bean Validation checks incoming DTOs.
- Duplicate usernames return HTTP 409.
- Missing customers/accounts return HTTP 404.
- Invalid credentials return HTTP 401.
- Insufficient balance or inactive account returns HTTP 409.
- GlobalExceptionHandler provides a consistent JSON error contract.

13. Testing Strategy

- JUnit service tests for deposit balance changes.
- Withdrawal test for insufficient funds.
- Authentication and authorization scenarios.
- Customer/account API validation.
- Postman verification of HTTP status codes and JSON responses.

16. CV-Safe Contribution

Implemented Controller-Service-Repository architecture using Java, Spring Boot, REST APIs and MySQL; developed customer/account APIs and banking business logic for deposits, withdrawals and transaction history; implemented JWT-based authentication and authorization; added validation, exception handling and JUnit testing.